PROYECTO TLP02-2026 / INFORME TÉCNICO / FASE 1

Gramáticas formales y lenguaje natural

Un parser LL(1) compartido para un subconjunto de C++ y un español controlado

Autor: Alan Ernesto Espinales Lovo

Carné: 00543024

Asignatura: Teoría de Lenguajes de Programación · 7 de noviembre de 2026

Resumen

Se implementó un analizador predictivo LL(1) en Python para un subconjunto de C++ y se reutilizó el mismo motor con un lexer y una gramática de español. La extensión controla algunas concordancias de género y número, cláusulas relativas de sujeto y modificadores. Para investigar sus límites se ejecutaron pruebas de aceptación, rechazo y casos deliberadamente problemáticos. El proyecto incluye 33 oraciones comparadas con spaCy y una auditoría adicional de 49 entradas, así como una ampliación del vocabulario español. Los resultados muestran que el parser responde de forma consistente dentro de su gramática, pero rechaza construcciones válidas que no fueron programadas. Se distinguen esas restricciones de los límites propios del método LL(1) y de las gramáticas libres de contexto (CFG).

1. Introducción y objetivos

Los lenguajes de programación tienen una sintaxis diseñada para interpretarse sin ambigüedades innecesarias. El español es mucho más flexible: admite variaciones de orden, construcciones subordinadas, modificadores y expresiones con más de una interpretación. El proyecto propone examinar esa diferencia mediante una experiencia concreta: desarrollar primero un parser formal y luego aplicarlo al lenguaje natural [1].

El objetivo práctico fue construir un solo programa capaz de analizar dos configuraciones lingüísticas. El objetivo experimental fue localizar qué regla provoca cada rechazo y establecer si el problema nace del lexer, de las producciones que elegimos, del análisis predictivo LL(1) o de una limitación formal más amplia. El estudio no busca interpretar todo el español ni generar código ejecutable a partir de C++.

Diseño de las pruebas

Fuente de evidencia	Alcance	Cómo se interpretó
Pruebas automáticas	84 pruebas de Python; todas aprobadas	Comportamiento del código actual.
Comparación NLP	33 oraciones; 20 aceptadas y 13 rechazadas por LL(1)	Se comparan con las anotaciones conservadas de spaCy.
Auditoría de límites	49 entradas adicionales: 36 español y 13 C++	Se estudian errores reales y rechazos esperados.

Estos conjuntos cumplen funciones distintas y no deben sumarse como si fueran una única prueba de precisión del español.

2. Marco teórico

2.1. Lenguajes formales y lenguaje natural

Un lenguaje formal se define por símbolos y reglas explícitas: la sintaxis decide qué cadenas pertenecen al lenguaje. En cambio, el lenguaje natural presenta polisemia, ambigüedad estructural, concordancias, elipsis y variación de orden. También tiene recursividad: una construcción puede contener otra de la misma clase. Un árbol sintáctico describe cómo se agrupan los componentes, pero no garantiza por sí solo que una expresión tenga sentido en el mundo.

2.2. Jerarquía de Chomsky

Nivel	Tipo de gramática	Idea esencial
Tipo 3	Regular	Patrones finitos y autómatas finitos.
Tipo 2	Libre de contexto (CFG)	Reglas A → α; describen muchas estructuras anidadas.
Tipo 1	Sensible al contexto	Permite restricciones formales que las CFG no expresan.
Tipo 0	Sin restricciones	Clase más general de la jerarquía.

Nuestro parser utiliza gramáticas de tipo 2. El hecho de que su análisis sea LL(1) añade una condición: con un único token de anticipación debe poder elegirse una sola producción en cada celda de la tabla [2]. No toda CFG es LL(1), aunque muchas pueden transformarse para facilitar el análisis predictivo.

2.3. CFG, LL(1) y significado no son lo mismo

- **CFG:** conjunto de reglas que define estructuras posibles, incluidas muchas construcciones recursivas y concordancias con categorías gramaticales finitas.
- **LL(1):** estrategia determinista que selecciona una producción mediante FIRST, FOLLOW y una tabla predictiva. Si dos alternativas compiten en la misma celda, esa formulación no es LL(1).
- **Léxico y semántica:** el lexer clasifica las palabras; el parser ordena tokens; comprobar si una frase tiene un significado razonable requiere información adicional.

2.4. Por qué es importante la ambigüedad

La oración «El niño vio a la niña con el telescopio» puede describir el instrumento usado para mirar o una característica de la niña. Una CFG puede representar varias derivaciones para una oración; la dificultad es decidir cuál interpretación corresponde al contexto [3]. Nuestro programa no resuelve ese ejemplo: lo rechaza antes de construir el árbol porque «vio» queda fuera del reconocimiento implementado.

3. Implementación del parser

3.1. Arquitectura compartida

La aplicación tiene una única entrada, main.py, y un motor común, parser_ll1.py. Para cada modo utiliza un lexer y una gramática propios: lexer_cpp.py/gramatica_cpp.py o lexer_es_v3.py/gramatica_es_v3.py. El motor calcula FIRST y FOLLOW, construye la tabla LL(1), procesa tokens mediante una pila y produce un árbol de derivación. Los errores incluyen línea, columna, token encontrado y, cuando corresponde, tokens esperados.

```
Entrada de texto → Lexer del modo seleccionado → Tokens
Tokens + gramática → Tabla LL(1) / pila → Árbol o error
```

3.2. Subconjunto de C++

La gramática contiene 17 no terminales y 35 producciones. Reconoce declaraciones int/float, inicializaciones, asignaciones, expresiones con precedencia, comparaciones y condicionales if/else con llaves. No incluye funciones, ciclos, clases ni comprobación semántica de variables. Las llaves hacen explícito el alcance de las ramas de if/else.

```
Declaracion → Tipo ID Inicializacion SEMI
Inicializacion → ASSIGN Expresion | ε
Condicional → IF LPAREN Condicion RPAREN Bloque ElseOpcional
Expresion → Termino RestoExpresion
```

Entrada de C++	Respuesta verificada	Interpretación
int edad = 20;	Aceptada	Declaración válida dentro del subconjunto.
int edad = ;	Rechazada en Expresion	Falta un valor luego de «=».
int while = 5;	Aceptada indebidamente	El lexer trata una palabra reservada como ID.

3.3. Gramática implementada para español

La gramática española del programa tiene 21 no terminales y 40 producciones; el diccionario reconoce 1,679 formas escritas. Se mantiene el mismo motor LL(1) usado para C++. El análisis distingue sujeto masculino/femenino y singular/plural (MS, FS, MP, FP), exige concordancia entre artículo y sustantivo, y selecciona verbo singular o plural según el sujeto. La ampliación del vocabulario no modifica estas reglas.

```
Oracion → SujetoMS PredicadoS PuntoOpcional EOF | ...
SujetoMS → ART_MS SUST_MS AdjetivoMSOpc RelativaSOpc
RelativaSOpc → QUE VERBO_S AdverbioOpc FrasePrepOpc | ε
PredicadoP → VERBO_P AdverbioOpc FrasePrepOpc
```

3.4. Qué oraciones acepta: alcance del subconjunto

El parser no reconoce español libre. Acepta oraciones enunciativas con sujeto explícito antes del verbo y esta estructura general (los corchetes indican elementos opcionales):

```
Oración = Sujeto + Predicado + [punto]
Sujeto = artículo + sustantivo + [adjetivo] + [relativa de sujeto]
Predicado = verbo + [adverbio] + [frase preposicional]
Relativa = que + verbo + [adverbio] + [frase preposicional]
```

El artículo puede ser definido (el, la, los, las) o indefinido (un, una, unos, unas); los verbos del diccionario están en tercera persona singular/plural. La frase preposicional contiene preposición + artículo + sustantivo y admite un adjetivo opcional. La relativa es de sujeto y concuerda en número con su antecedente. La entrada solo se acepta si todas sus palabras están en el vocabulario y respetan ese orden.

Construcción reconocida	Ejemplo aceptado
Sujeto + verbo; artículo definido o indefinido	El carro corre. / Una estudiante estudia.
Adjetivo y adverbio opcionales	La niña pequeña canta alegremente.
Frase preposicional	El niño corre en el parque.
Relativa de sujeto con modificadores	Los niños que juegan en el parque corren rápidamente.

Quedan fuera del subconjunto los sujetos con nombres propios o sujetos coordinados, el orden verbo–sujeto, los objetos directos, varios adjetivos seguidos, la coordinación verbal y las relativas anidadas. Por ejemplo, «Diego ama la pizza» y «El niño y la niña juegan pelota» son correctas en español, pero son rechazadas por nuestra gramática. Estas exclusiones son limitaciones de cobertura, no de todas las CFG.

4. Experimentos: ¿dónde falla el español?

Se buscaron oraciones gramaticalmente válidas que el parser no reconoce. La tabla señala el punto exacto donde se detuvo el motor. Un rechazo así evidencia un límite de cobertura del subconjunto; no demuestra por sí mismo una imposibilidad de todas las CFG.

Entrada válida	Respuesta / punto de rechazo	Límite identificado
Diego ama la pizza.	Oracion ante DESCONOCIDA («Diego»).	Nombre propio sin artículo y objeto directo no incluidos.
El niño y la niña juegan pelota.	AdjetivoMSOpc ante DESCONOCIDA («y»).	Sujeto coordinado y objeto directo no implementados.
Corre el niño.	Oracion ante VERBO_S.	Solo se implementó orden sujeto–verbo.
La niña pequeña tranquila canta.	RelativaSOpc ante ADJ_FS.	Solo un adjetivo posnominal en esa posición.
El niño corre en el parque con la niña.	AdjetivoMSOpc ante PREP («con»).	Solo una frase preposicional consecutiva.
El niño que juega con la niña que canta corre.	PredicadoS ante QUE.	No permite la relativa anidada en el complemento.
El niño que la niña conoce corre.	RelativaSOpc: esperaba VERBO_S, halló ART_FS.	Admite relativa de sujeto, no relativa de objeto.
La niña corre y canta.	AdverbioOpc ante DESCONOCIDA («y»).	Coordinación de verbos ausente.

4.1. Dos rechazos analizados paso a paso

Caso de nombre propio: «Diego ama la pizza». El lexer entrega DESCONOCIDA para «Diego» y el no terminal Oracion solo posee alternativas que comienzan con ART_MS, ART_FS, ART_MP o ART_FP. Incluso agregando las palabras al diccionario, faltaría permitir un sujeto sin artículo y un verbo seguido de objeto directo. Es una limitación léxica y de gramática; no de la clase CFG.

Caso de anidación: «El niño que juega con la niña que canta corre». La primera relativa puede analizarse, pero el sintagma nominal dentro de «con la niña» no admite otra relativa. Cuando aparece el segundo «que», el motor intenta PredicadoS y falla. Una CFG sí puede tener reglas recursivas; aquí simplemente no se incluyeron en ese lugar.

4.2. Lo que sí se comprueba correctamente

«Los niños que juegan con la niña corren» fue aceptada. En cambio, «Los niños que juegan con la niña corre» se rechazó en PredicadoP, que esperaba VERBO_P. Asimismo, «La niña pequeño canta» se rechazó por falta de concordancia del adjetivo. Estos son controles útiles: el parser puede verificar cierta concordancia separada por otras palabras, y un rechazo de una oración incorrecta no cuenta como defecto.

5. Análisis formal de los límites

5.1. Tres causas que no deben confundirse

Nivel del problema	Evidencia del proyecto	Conclusión razonable
Léxico o cobertura	«Diego» no recibe un token admitido.	Falta vocabulario o categoría de nombre propio.
Producciones del subconjunto	No hay segunda relativa ni dos adjetivos consecutivos.	Se necesitan reglas adicionales; una CFG podría expresarlas.
Decisión LL(1)	Dos reglas de Oracion empiezan con ART_FS SUST_FS.	Se genera conflicto predictivo en la misma celda; podría factorizarse.

El motor detectó un conflicto al probar las alternativas «Oracion → ART_FS SUST_FS VERBO_S EOF» y «Oracion → ART_FS SUST_FS ADJ_FS VERBO_S EOF»: ambas buscan la celda (Oracion, ART_FS). Se resuelve introduciendo el prefijo común y una regla opcional para adjetivo. Por tanto, el conflicto afecta esa formulación LL(1), no demuestra que la estructura deje de ser libre de contexto.

5.2. ¿Qué limitación sí pertenece a la clase CFG?

En teoría de lenguajes, el lenguaje $L = \{a^n b^n c^n \mid n \geq 1\}$ exige igualar tres cantidades no acotadas y no es libre de contexto. Sirve para ilustrar que existen dependencias fuera del poder expresivo de una CFG de tipo 2. Sin embargo, este es un ejemplo matemático, no una prueba obtenida al analizar las oraciones españolas del proyecto. En nuestro subconjunto, las concordancias de género y número tienen valores finitos y pueden codificarse mediante categorías gramaticales.

5.3. Ambigüedad y semántica

La frase del telescopio tiene más de una lectura estructural; una gramática ambigua podría generar más de un árbol, mientras que nuestro motor LL(1) exige una elección única en su tabla. No atribuimos el rechazo observado a la ambigüedad porque antes falla con «vio». De manera complementaria, «El telescopio duerme» es aceptada: satisface las reglas sintácticas aunque su lectura literal resulte extraña. El significado no es una propiedad evaluada por este parser.

5.4. Hallazgo adicional en C++

La auditoría comprobó que «int while = 5;», «int class = 2;» y «int return = 1;» son aceptadas. El lexer reconoce como reservadas solamente algunas palabras del lenguaje y clasifica otras como ID. Este sí es un error léxico respecto a C++ estándar, distinto de rechazar una función main(), que simplemente queda fuera del subconjunto declarado. Se deja documentado para evitar presentar esas aceptaciones como correctas.

6. Comparación con spaCy

Se usaron las mismas 33 oraciones de la batería original con el parser unificado y spaCy 3.8.18, modelo es_core_news_sm. El archivo JSON conserva las anotaciones de spaCy de la ejecución del 7 de noviembre de 2026; tras ampliar el diccionario se actualizaron solamente los diagnósticos del parser, no se volvió a ejecutar el modelo. spaCy devuelve etiquetas morfológicas y relaciones de dependencia; no un dictamen automático de corrección gramatical [4].

Oración (ID)	LL(1)	Salida relevante de spaCy
La niña canta. (E02)	Acepta; «canta» es VERBO_S.	Marca «canta» como ADJ y «niña» como raíz: error de etiquetado.
El niño que juega corre. (E15)	Acepta la relativa de sujeto.	Relaciona «juega» con «niño» mediante acl y «corre» como raíz.
La niña pequeño canta. (E24)	Rechaza ADJ_MS en AdjetivoFSOpc.	Da Fem a «niña», Masc a «pequeño» y aun así genera dependencias.
Corre el niño. (E30)	Rechaza el orden verbo–sujeto.	Identifica «Corre» como raíz verbal y «niño» como nsubj.
La niña corre y canta. (E31)	Rechaza la coordinación.	Relaciona «canta» con «corre» como conj.
El niño vio a la niña con el telescopio. (E33)	Rechaza «vio».	Relaciona «telescopio» con «vio»; elige una interpretación, no las dos.

6.1. Lectura crítica de los resultados

En las 33 entradas, el parser aceptó las 20 previstas dentro de su gramática y rechazó las otras 13: siete con errores de concordancia y seis oraciones válidas fuera del alcance. Esto describe consistencia respecto a las etiquetas esperadas para el subconjunto, no una precisión del 100 % sobre español libre. spaCy procesó las 33 entradas, pero «procesar» tampoco equivale a «analizar correctamente»; por ejemplo, clasificó «canta» como adjetivo en E02 y como verbo en E19.

Las salidas evidencian diferencias de propósito: nuestro parser verifica el cumplimiento de reglas explícitas y detiene el análisis ante una estructura no prevista; spaCy intenta asignar categorías y relaciones incluso a oraciones incorrectas o fuera de nuestro vocabulario. Las predicciones de spaCy pueden contener errores y necesitar revisión humana.

6.2. Alcance de la comparación

No se calculan métricas comparables de exactitud entre sistemas porque sus salidas y objetivos son diferentes: uno produce aceptación y un árbol de constituyentes; el otro, etiquetas y dependencias. Las 49 pruebas adicionales de la auditoría no se pasaron por spaCy. Por ello, los ejemplos nuevos, incluido «Diego ama la pizza», se reportan únicamente como resultados del parser.

7. Conclusiones y reproducibilidad

7.1. Conclusiones

- **El motor sí se reutilizó.** Una sola implementación LL(1) analiza C++ y un subconjunto de español mediante lexers y gramáticas diferentes. El árbol de derivación y los errores permiten inspeccionar decisiones concretas.
- **La concordancia básica es manejable.** Las reglas de género y número aceptan ejemplos previstos y rechazan varias discordancias, incluso cuando sujeto y verbo están separados por una relativa simple.
- **El principal límite observado es de cobertura.** Nombres propios sin artículo, sujetos coordinados, objetos directos, orden invertido y ciertas subordinadas no forman parte de las producciones implementadas; extender vocabulario por sí solo no resuelve esas ausencias.
- **Las limitaciones formales requieren cuidado.** Un rechazo de este parser no prueba que toda CFG sea insuficiente. El conflicto LL(1), la ambigüedad estructural y la incapacidad de expresar ciertas dependencias no acotadas son asuntos diferentes.
- **spaCy complementa, no sustituye, el experimento.** Analiza mayor variedad de estructuras, pero sus etiquetas son predicciones y algunas fallan. La comparación permite contrastar reglas deterministas con un modelo entrenado.

7.2. Cómo reproducir la evidencia

En una terminal ubicada en la raíz del proyecto, con Python 3.13, ejecutar:

```
py -3.13 main.py
py -3.13 -m unittest discover -s pruebas -p "pruebas_*.py" -v
py -3.13 comparar_spacy.py --solo-parser
```

La ejecución completa de pruebas automatizadas se obtiene mediante el comando indicado en README.md e incluye 84 pruebas. Para observar fallos concretos, abrir desde el menú de español el archivo ejemplos/casos_fallas_es.txt. La evidencia principal está en evidencias_fallos/Resultados_pruebas_limites_TLP.csv y resultados_comparacion/comparacion_spacy.json. Para regenerar la comparación completa se requiere spaCy con el modelo de español instalado.

Referencias

[1] Proyecto TLP02-2026. «Gramáticas formales y lenguaje natural: experimentando los límites de la jerarquía de Chomsky». Guía de la asignatura, Fase 1, 2026.

[2] Cornell University, CS 4120. «Context-Free Grammars» y «Top-down (LL) Parsing». https://www.cs.cornell.edu/courses/cs4120/2026sp/notes/

[3] Bird, S.; Klein, E.; Loper, E. Natural Language Processing with Python, cap. 8, «Analyzing Sentence Structure». https://www.nltk.org/book/ch08.html

[4] spaCy. «Spanish · Trained pipelines» (es_core_news_sm) y documentación de análisis de dependencias. https://spacy.io/models/es